这一讲：不用从头再来



第三部分

1 表示：反复出现的结构，存成文字还是代码?

第三部分

2 归纳：怎么从自己做过的事里提炼出来?

第四部分

3 生命周期：什么时候取用、修改、忘掉?

上一讲的问题 存什么 · 存在哪 第一部分 什么时候读回来 第二、三部分 什么时候出错 第三部分末尾 + 第四部分



课件 p.2

任务里有大量结构在反复出现

任务 1
把索尼耳机
加进心愿单

任务 2
查无线键盘的
价格区间

共享的搜索过程：
打开商店 → 找搜索框 → 填词 "sony headphones" → 点搜索

各自收尾：
任务 1：打开索尼那条 → 加入心愿单
任务 2：读出价格 → 报告最低到最高

? 原始动作，还是函数?

? 哪部分值得复用?

? 矛盾了，改还是删?



课件 p.2（中文重绘）

经验能存在哪？三个位置

上下文窗口

上一讲



优点 保真度高

缺点 贵、噪声大，不会判断什么重要

外部产物

这一讲



文件系统 · 数据库

优点 可查看、可编辑、可检索、可移植

缺点 要归纳、挑选和维护

模型权重

后面 SFT 那一讲



优点 推理快，能大范围改变行为

缺点 更新慢、不透明，只对这个模型有效

跨模型移植

模型 A　模型 B　模型 C → 怎么在这个网站上搜商品

人和系统之间的接口

写最佳实践 → ← 人再改　说明文档　→ 照着做 ← 先起草

哪些经验值得变成外部产物？



课件 p.3（中文重绘）

同一个子任务会做不止一次，技能才有用

① 片段

完整轨迹：每个观察和动作都在



什么时候有用

以后还要用到当时的商品、价格或操作顺序

② 事实

一条以后还用得上的陈述

“这家店只在商品详情页显示价格”

记忆设置（示意）

住在……

喜欢……

最近在做……

模型从你过去的对话里生成

什么时候有用

这个事实以后还会出现，或会被更新

③ 技能

怎么完成任务的一部分，和这次的细节无关

填搜索框 → 点搜索 → 打开一条结果

文字	找搜索框，填词，点搜索
文字 + 动作	`fill(q) → click(搜索)`
代码	`def search(q): ...`

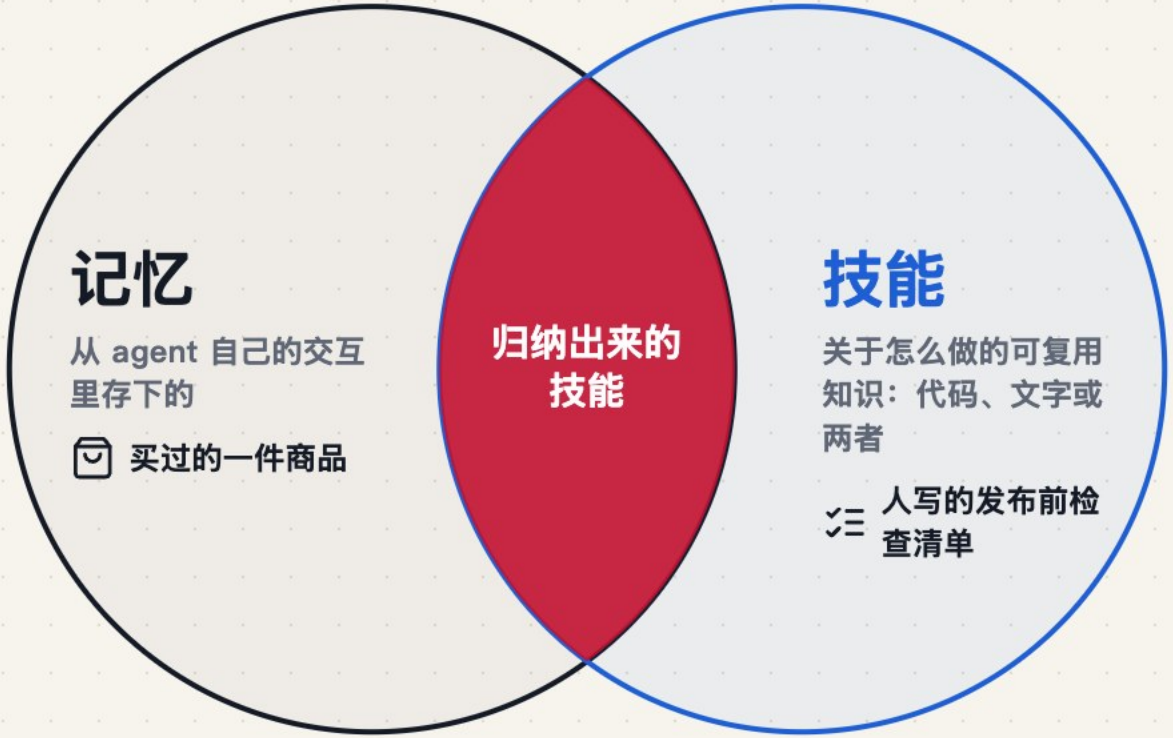
什么时候有用

以后的任务会重复同一个模式



课件 p.4（中文重绘）

记忆和技能：画两个圈



agent 从自己成功的搜索里归纳 · 前提：判断那次确实成功了



课件 p.5–6（中文重绘）

技能的两条来路

	技能作为指令	技能作为学习
作者	人或组织	agent 自己
来源	专业经验、规章、文档	它自己的片段和结果
长处	意图和出处清楚	适合只能靠摸索发现的技能 · 能自动化
主要负担	专家花时间写和维护	判断对错、验证、筛选 还可能有噪声

agent 提出候选 → 人来改 → 来回几轮

⇄ 互补

人先写 → agent 根据经验细化



课件 p.5–6（中文重绘）

让模型写技能：常见的坑

常见毛病

写太多、推断过头





黑块：你想要的　红框：多出来的（示意）

做的远比你想要的多

一个做法

拿三四张同类工单，让它写



一份技能

别的 agent 能照着做

实践中常见

不太会设身处地，想读的人怎么理解





提示它这么想，效果好很多



课件 p.5–6（中文重绘）

同样的要求每次都敲，不如存成技能

PR 对话（示意） 第 3 次

请按这些要求检查这段 Python 代码：

- 用 `Black` 格式化，每行不超过 88 个字符
- 用 `Ruff` 做 lint（我们的自定义规则）
- 公开 API 都写类型标注
- 用 Google 风格的 docstring
- 新函数都用 `pytest` 测



课件 p.7–8

同样的要求每次都敲，不如存成技能

PR 对话（示意）

第 3 次

一个文本文件

每次都一样的要求

→ 当指令

agent

按固定规范、反复做的事

加测试　评审 PR　升级依赖

agent 会结合情况理解，但要求不变

技能 = 一组文件

文档　脚本　资源



课件 p.7–8

python-review

示意 · 内容照课件 p.9

```
---
name: python-review
description: This skill should be used when the user asks to
  "review Python code", "check Python style", "lint Python",
  or mentions code quality.
triggers:
  - python review
  - code review
  - lint python
---
# Python Code Review
Review Python code using company standards...
## 步骤
1. 跑 scripts/run_checks.py: Black (88 字符) + Ruff
```



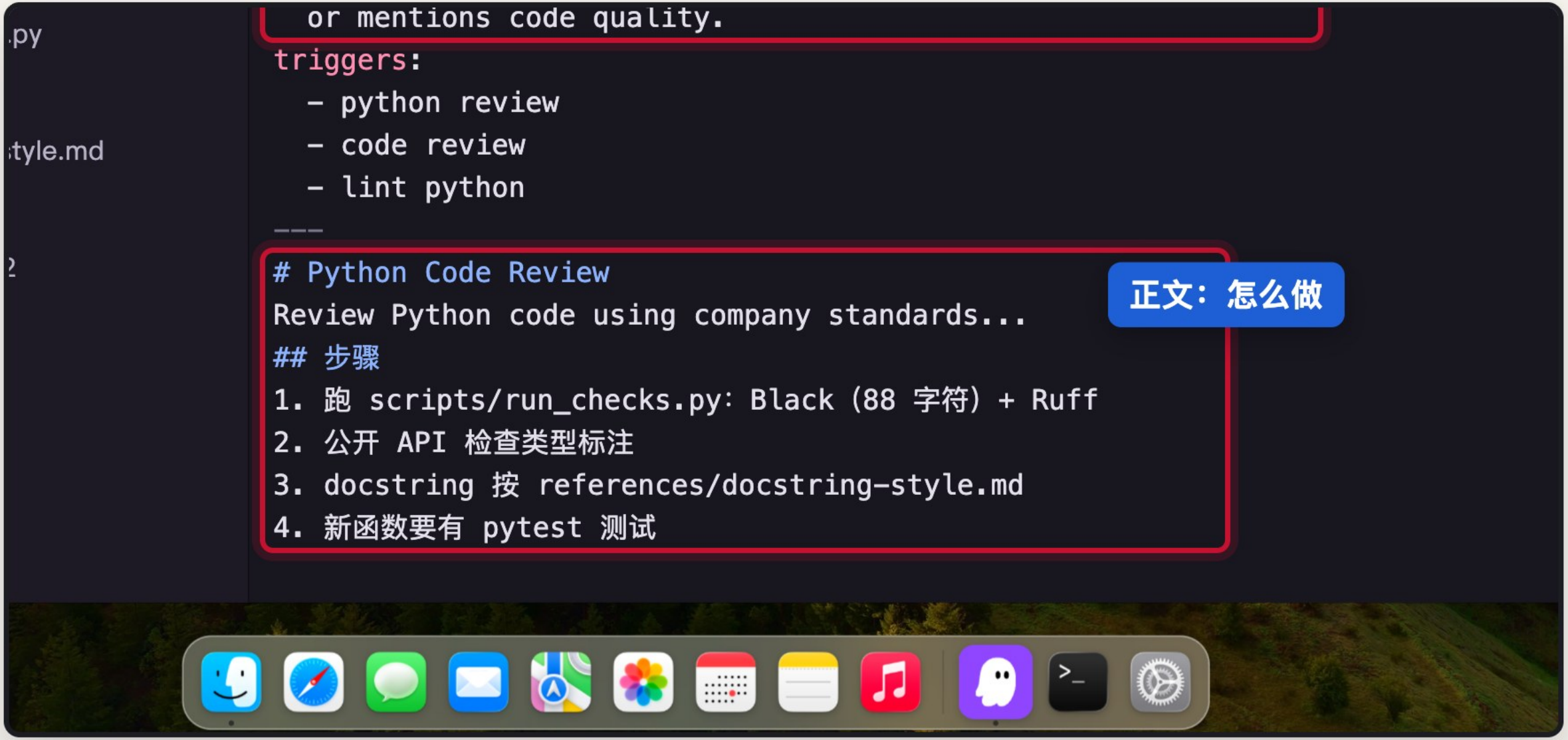
课件 p.9（窗口为示意）

.py

tyle.md

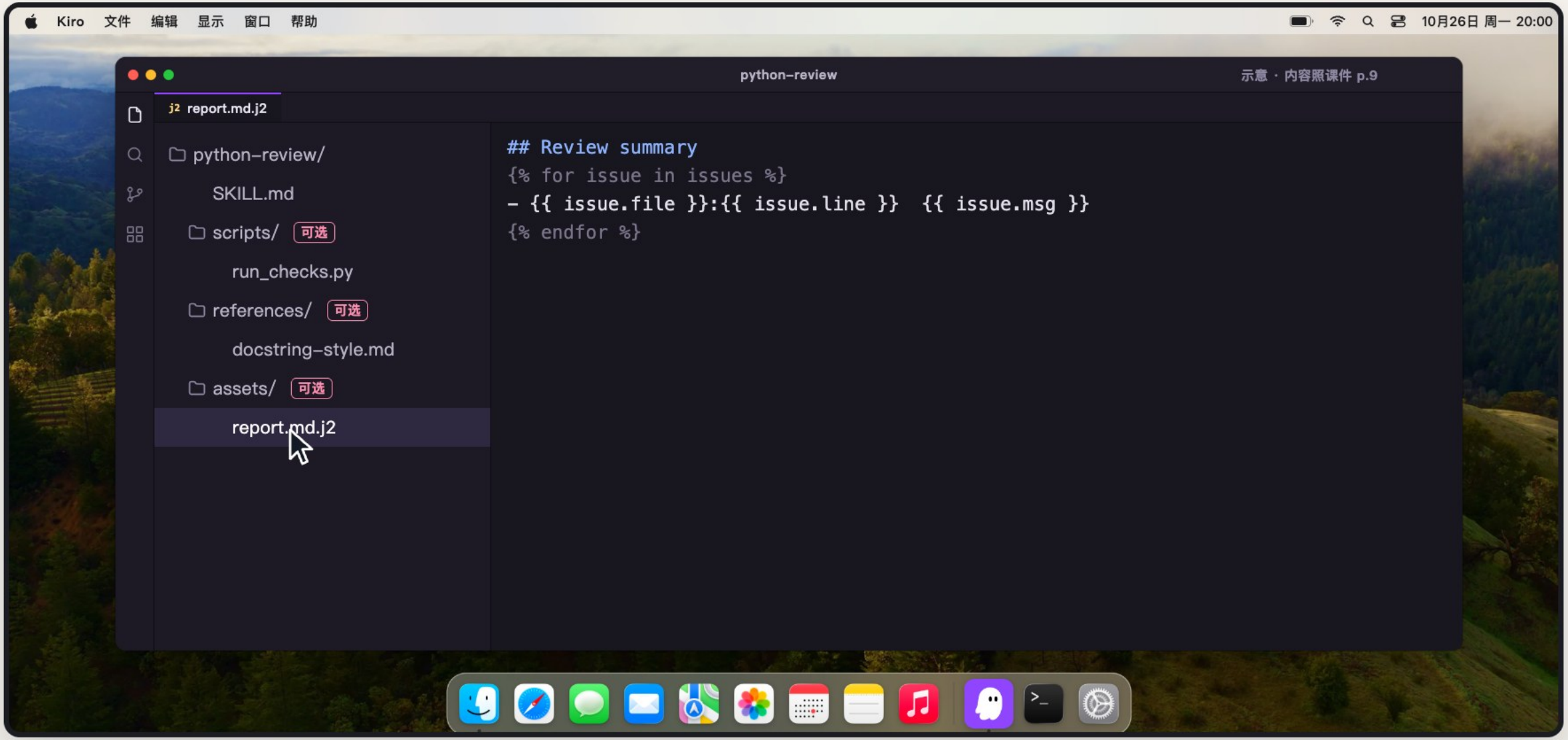
```
  or mentions code quality.
triggers:
  - python review
  - code review
  - lint python
---
# Python Code Review
Review Python code using company standards...
## 步骤
1. 跑 scripts/run_checks.py：Black（88 字符）+ Ruff
2. 公开 API 检查类型标注
3. docstring 按 references/docstring-style.md
4. 新函数要有 pytest 测试
```

正文：怎么做





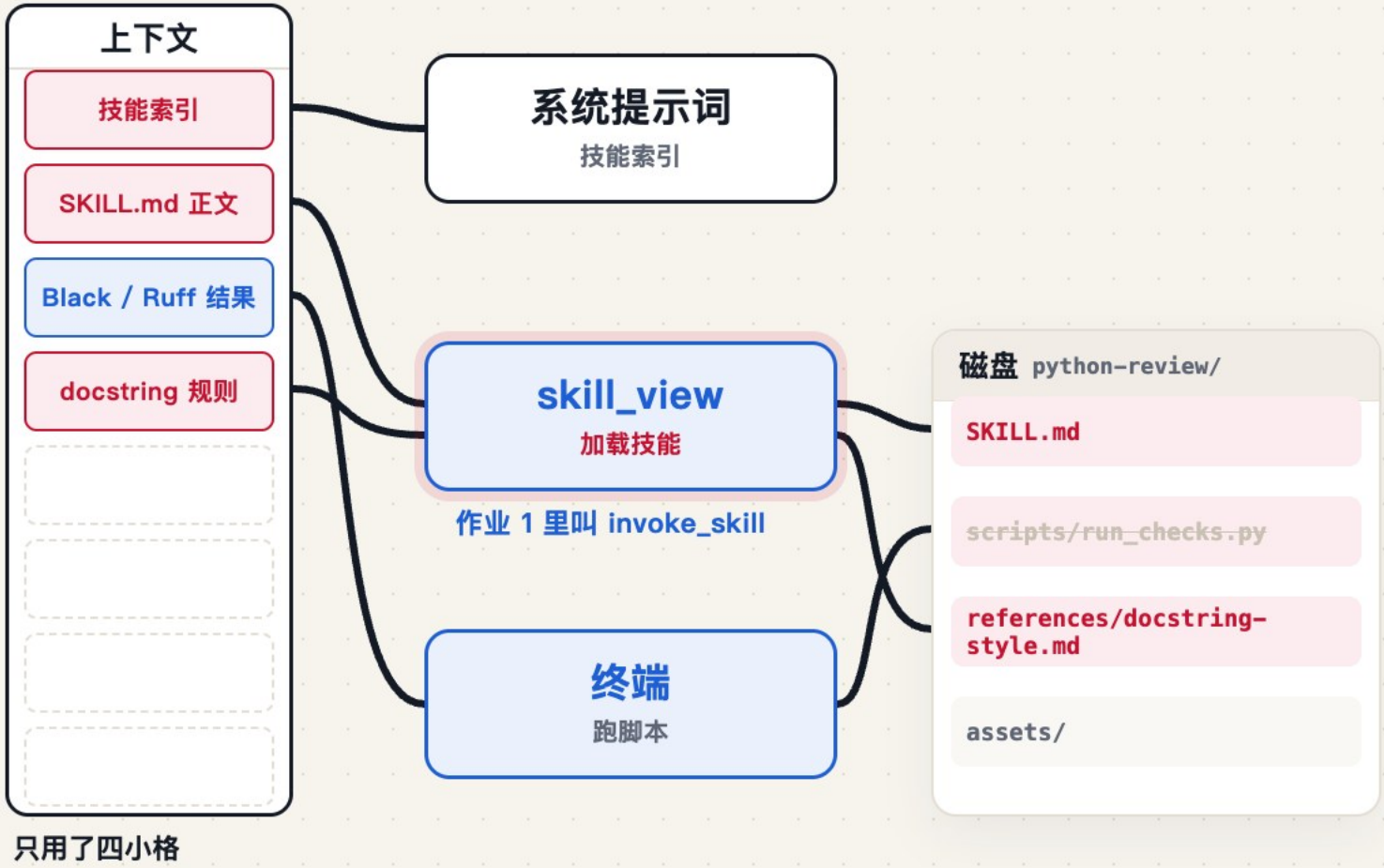
课件 p.9（窗口为示意）


Kiro 文件 编辑 显示 窗口 帮助
10月26日 周一 20:00
python-review
示意 · 内容照课件 p.9
j2 report.md.j2
python-review/
SKILL.md
scripts/ 可选
run_checks.py
references/ 可选
docstring-style.md
assets/ 可选
report.md.j2
Review summary
{% for issue in issues %}
- {{ issue.file }}:{{ issue.line }} {{ issue.msg }}
{% endfor %}



课件 p.9（窗口为示意）

渐进式披露：用到哪层，才加载哪层



三层 · 什么时候进上下文

1	SKILL.md 元数据（YAML）	一直加载
2	SKILL.md 正文	触发技能时
3	附带文件（文本、脚本、数据）	去读时

× 脚本代码不进上下文



课件 p.9–11（中文重绘）

几个常被问到的问题

课上口头补充

1 技能多了，会拖累吗？

系统提示词 · 技能索引

会 → 数量要控制

无关技能 无关技能 无关技能 无关技能

上千上万个技能的情况很少见

2 触发灵不灵？

触发 = 一次工具调用

```
skill_view("python-review")
```

调不调，取决于 模型本身 你写的描述

☑ 所以要评测

3 为什么是 SKILL.md、为什么是 Markdown？

SKILL.md = 标准名字　模型、框架都按它训练、加载

```
# 可视化技能
见 [示例](viz.html)
![图](chart.png)
```

× 难表达动态的东西

引用

viz.js · 可交互小应用

文字技能 → SKILL.md　程序技能 → scripts/

4 能从一大堆技能里检索吗？

向量检索

技能 技能 技能 技能
技能 技能 技能 技能
技能 技能 技能 技能

进某个网站

该站技能 该站技能 该站技能

按领域加载

据说生产系统也这么做



课上问答 · 口头补充

SkillsBench：三步建起来

① 构建

234,896 | 1,773,213 | 5,891

三个来源的技能数（按原图读数）

去重

2,014,000 个不同的技能

142 位贡献者 → 400 个任务

任务另外写，和技能分开

② 过滤

自动检查

- ✓ 结构
- ✓ Oracle 100%
- ✓ AI 检测
- ✓ 泄漏审计

人工审核

- ✓ 数据有效性
- ✓ 任务真实性
- ✓ Oracle 质量
- ✓ 技能质量
- ✓ 防作弊

8 个领域 · 87 个任务

检查项照原图翻译

③ 评测

无技能 ■■■ | 有技能 ■■■

pytest ● 通过 ● 失败

OpenHands | Claude Code | Codex CLI | Gemini CLI

18 种配置 · 9,396 条轨迹

每个任务两种条件 × 各跑 3 次

技能池 [技能] [技能] [技能] [技能] [技能] → 人来挑 → 任务

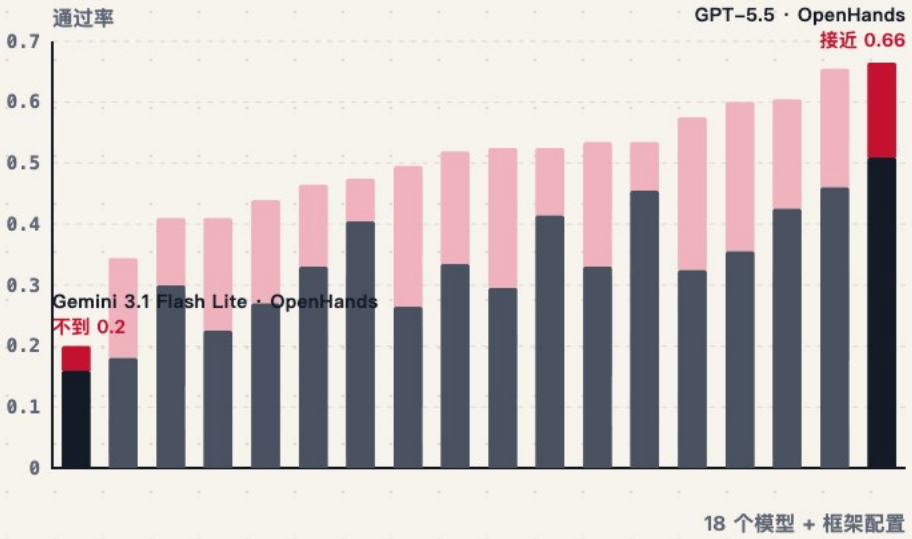
评的是：模型 + 框架能不能用好技能



课件 p.12

技能让通过率明显提高

33.9% → 50.5% 平均通过率



需要几个技能

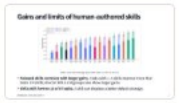


SKILL.md 长度



条长 = 提升多少，只画大小关系（示意）

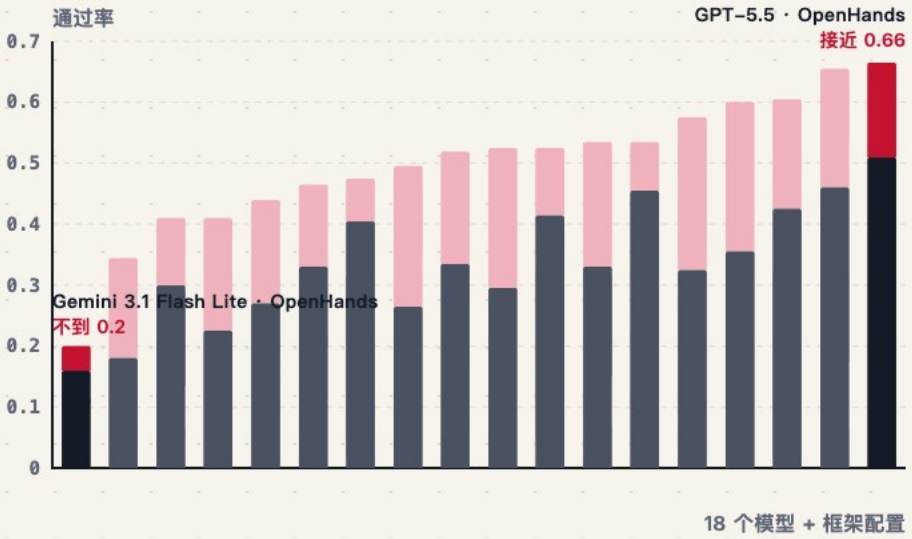
一部分来自难度，一部分是技能多了会分心

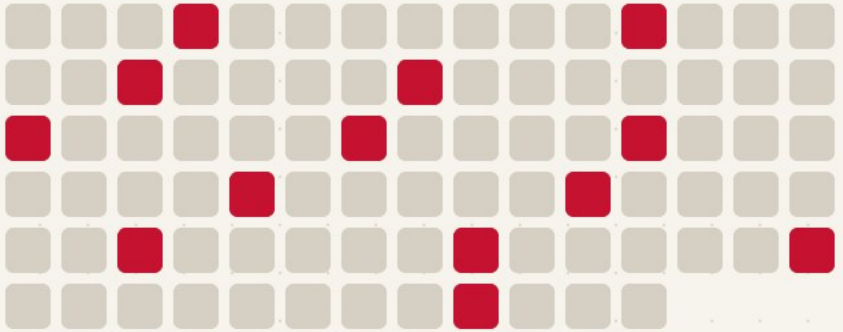


课件 p.13

87 个任务里，有 13 个变差

33.9% → 50.5% 平均通过率





87 个任务 · 红色 13 个变差（位置示意）

可能顶掉了更好的默认做法

写完要检查：结果到底有没有变好



课件 p.13

技能包从哪来

① 随框架自带

Bundled

作为 harness 默认的一套发布

harness

技能 技能 技能

② 安装或自己写

Installed / written

提交到仓库，或从别处装

.agents/skills/

别的仓库 · 技能市场

③ agent 自己创建

Created by agent

Hermes · skill_manage
新建 / 打补丁 / 删除

OpenHands · /skill-creator
从刚做完的工作流起草

✓ 人先审，再上线

OpenHands 的建议

别从零写。agent 刚做完一个想重复的工作流，马上创建，再多测几次。

只看任务描述，凭空写

模型来写

效果反而被拉低 ↓

（示意，不标数值）

真做一遍，再写

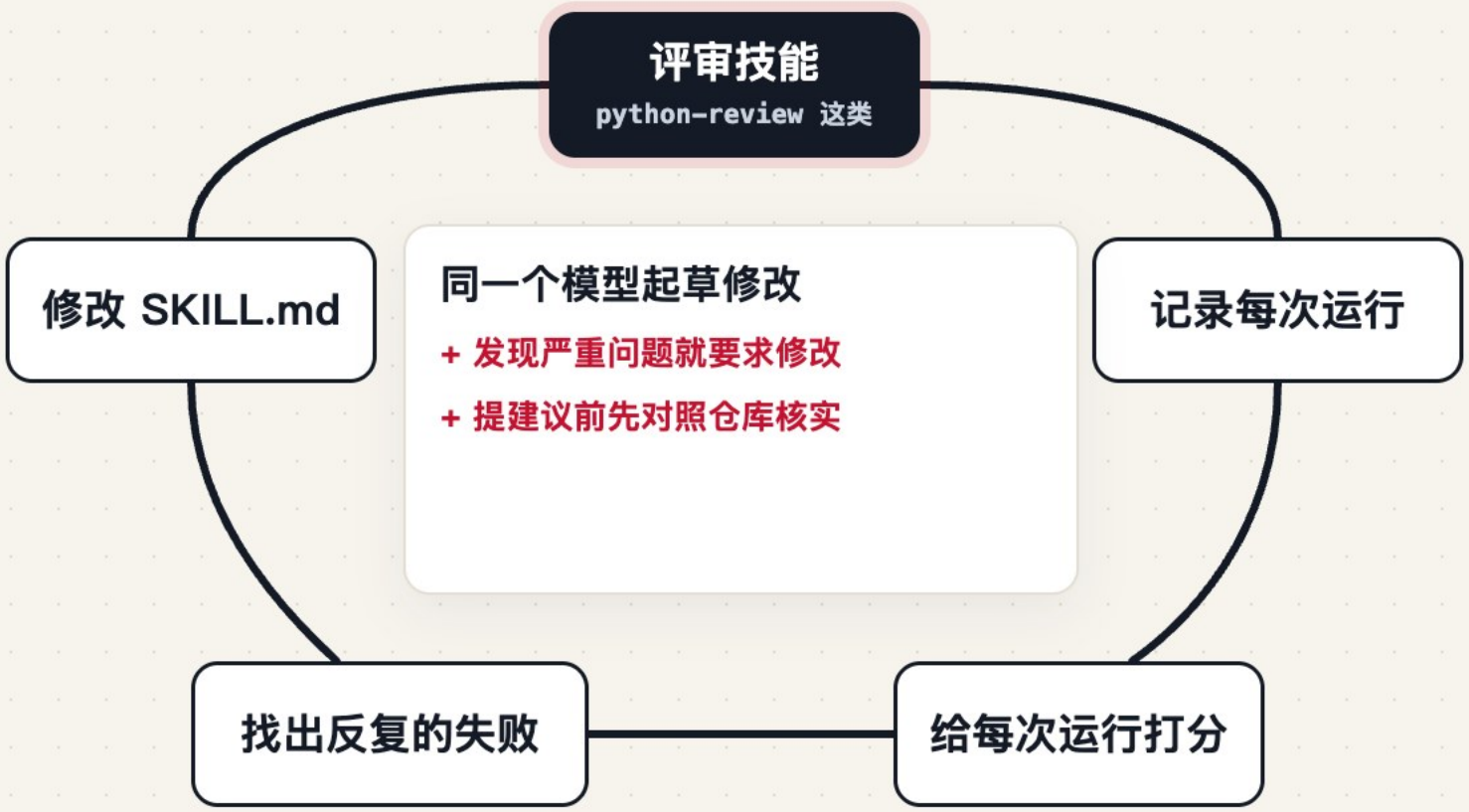
做过再写

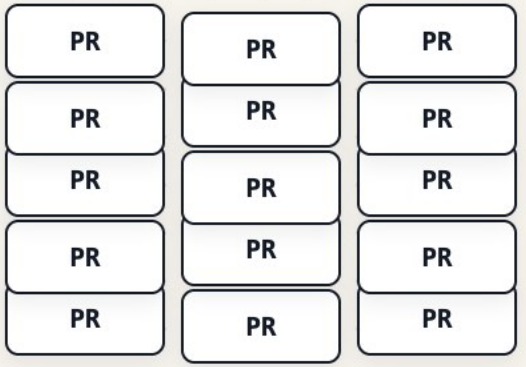
多出几条
事先没想到的要求



课件 p.14

维护一个代码评审技能：一个循环





几百、几千个 PR 的证据



课件 p.15

MemGPT：思路来自存储层次

自动记住过去的经验 → 从经验里归纳技能

↑ 越往上越快、越小

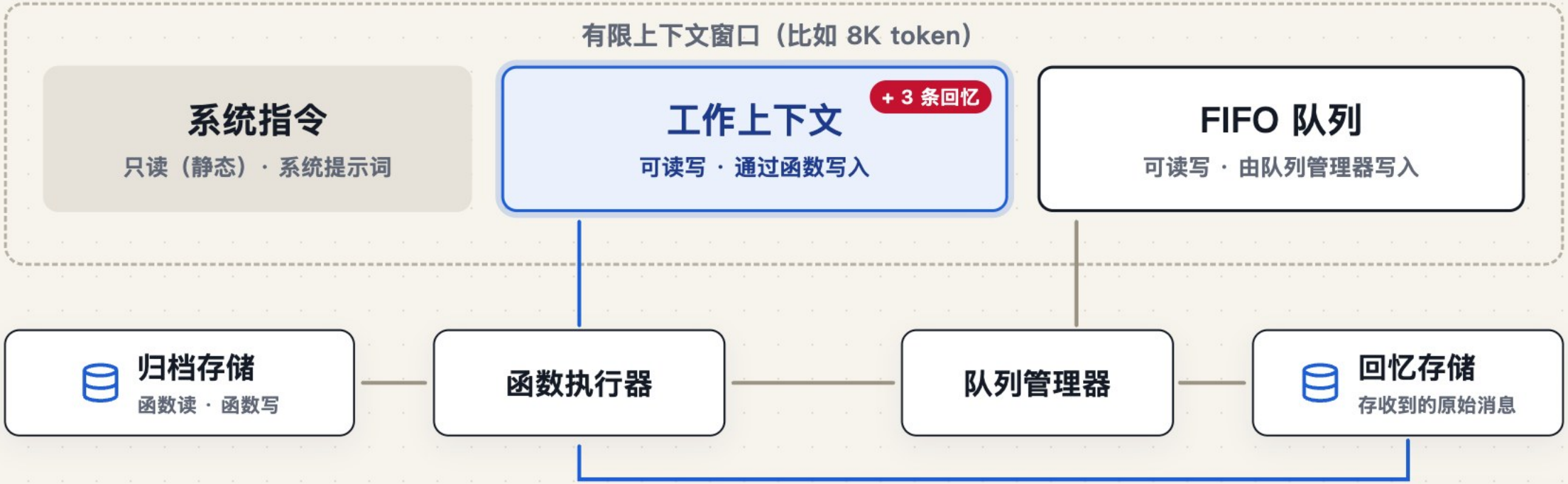


越往下越大、越慢 ↓



课件 p.16–17（中文重绘）

读和写，都是 agent 自己发出的函数调用



论文写时 窗口只有约 8K token，连聊天都需要它

现在 窗口大了，agent 要存的也更多

```
> recall_storage.search("six flags")
共 3 条结果（第 1/1 页）：
[2024-01-24] “哈哈对，six flags”
[2024-01-14] “我超爱 six flags，去过大概 100 次了”
[2023-10-12] “我和 james 其实是在 six flags 认识的”
```



课件 p.16–17（中文重绘）

Mem0：DeltaNet 在文字层面的对应物

① 提取

新消息 “我被裁了”

对话摘要

最近几条消息

大模型
提取候选记忆

候选记忆
用户被裁了

候选记忆
……

② 更新

相似的旧记忆（示意）

记忆库

向量检索 · 最相似的几条

喜欢爬山

养了一只猫

大模型
一次工具调用

ADD 新增
没存过相似的

UPDATE 更新
改变了旧记忆

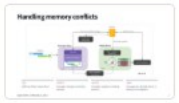
DELETE 删除
否定了旧记忆

NOOP 不动
存过，或无关

回链上一讲
线性注意力：只能往记忆里追加
DeltaNet：先取出 key 对应的值，再修改

Mem0 ≈ 它在文字层面的对应物

改权重这条路
遗忘学习（unlearning）



课件 p.18（中文重绘）

用自然语言说清哪里错了



人　裁判模型　→ 用自然语言说清哪里错了

同一个任务 ---------- 相似的任务

第 1 次尝试

“1776 年 10 月 28 日，在纽约州怀特普莱恩斯附近打的那一系列战役是什么？”

`Finish[Battle of White Plains]` ✕ 错

生成的反馈

“……问的是一系列战役，我只给了其中一场的名字……下次我会给出更多背景，比如所属战役的名字……”

第 2 次尝试

`Finish[The New York and New Jersey campaign]` ✓ 对



课件 p.19（中文重绘）

有点像给 agent 用的 RAG

训练时离线收集

干活时一路攒

事先手写

+ 三种来源还能组合

经验池

轨迹 1 ✓

轨迹 2 ✗ 批注

轨迹 3 ✓ 改写过

轨迹 4 ✗

轨迹 5 ✓ 批注

新任务的描述

查无线鼠标的价格区间

示意

向量 · 比相似度（条长示意）

上下文 · 少样本示例

轨迹 1 ✓

轨迹 3 ✓

也可以只在当前领域里检索

ExpeL 训练时收集经验 → 提取洞见 → 评测时召回

同类工作 Agent S Synapse ICAL



课件 p.20（中文重绘）

问题：动作里写死了元素编号

技能 搜索商品

显示 {query} 的结果

```
# 要找 {query}，我在商店的搜索框里输入它并提交。
fill('element145', {query})
click('element147')
```

技能 报告价格区间

{query} 的价格区间是多少?

复用 ↑ 搜索商品

```
# 要报价格区间，我先在商店里搜索它。
fill('element145', {query})
click('element147')
```

新加的步骤

```
# 结果卡片上都标着价格，我读出最低和最高。
send_msg_to_user("{low} to {high}")
```

同一页面，只是编号变了

element145 → element212 示意

→ 改动作空间来应对（讲 GUI agent 时再说）

换一个界面不同的网站

网站 A → 网站 B：搜索界面完全不同

× 一定会坏



课件 p.22（中文重绘）

把行为抽象成函数：早有前史

Voyager

Wang et al., 2023



在 Minecraft 里归纳越来越复杂的函数，后面的调用前面学会的

DreamCoder

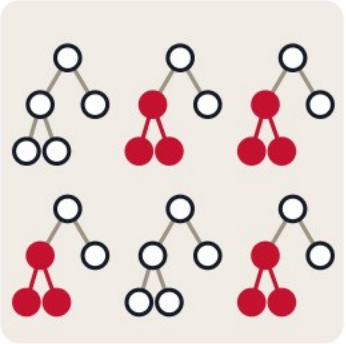
Ellis et al., 2021



从已解出的程序里找反复的模式，抽成函数，不断压缩程序

Stitch

Bowers et al., 2023



快速的符号压缩：找出共享结构最多的函数

LAPS

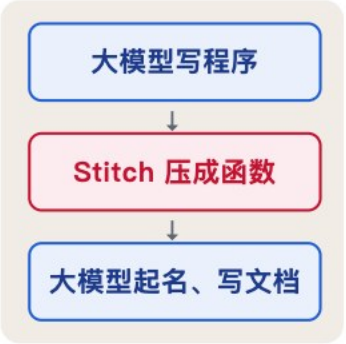
Wong et al., 2021



用自然语言描述引导：归纳哪些函数、怎么搜程序

LILO

Grand et al., 2024



三步接力：写程序、压缩、起名和写文档

→ 后来用到 agent 的技能归纳上



课件 p.23（中文重绘）

在线归纳：边做任务，边攒技能

设定：任务一个接一个来；成功了，先更新记忆，再接下一个

1 把索尼蓝牙耳机加进心愿单 ✓

① 成功后归纳

2 找上个月新上架的游戏配件 ✓

① 成功后归纳

② 之后应用

3 无线键盘的价格区间是多少? ✓

① 成功后归纳

② 之后应用

技能记忆 随评测一路增长

- 搜索商品
- 加入心愿单
- 报告价格区间

累计成功率

看它能不能越来越高（示意）



课件 p.24–25（中文重绘）

后面的论文，各落在这个循环的不同环节

学 LEARN：片段 → 裁判 → 归纳 → 准入 → 记忆

用 USE：记忆 → 选择 / 检索 → 行动 → 结果

维护 MAINTAIN：结果 → 新增 → 修复 → 淘汰 → 记忆

技能太多 → 精简：没人用的删掉

常用技能



太具体 → 泛化或合并

搜索 {商品}



课件 p.24–25（中文重绘）

归纳提示词的四条要求

片段 网页任务

查询

告诉我到目前为止，店里收到的评论中，有多少条提到“satisfied”?

动作

```
click("Marketing")
click("All Reviews")
fill(757, "satisfied")
click("Search")
send_msg_to_user("2")
```

大模型

少样本提示归纳

satisfied → {term}

工作流 归纳出的文字技能

任务：搜索匹配“{term}”的评论

```
# 要找 {term} 的评论，我在评论页搜索它
click("All Reviews")
fill(757, "{term}")
click("Search")
```

裁判：最终状态预测为正确?

✓ 是 → 存进记忆　× 否 → 不存

记忆

工作流
搜索匹配 {term} 的评论

工作流
……

全放　只放描述

↑ 放进上下文

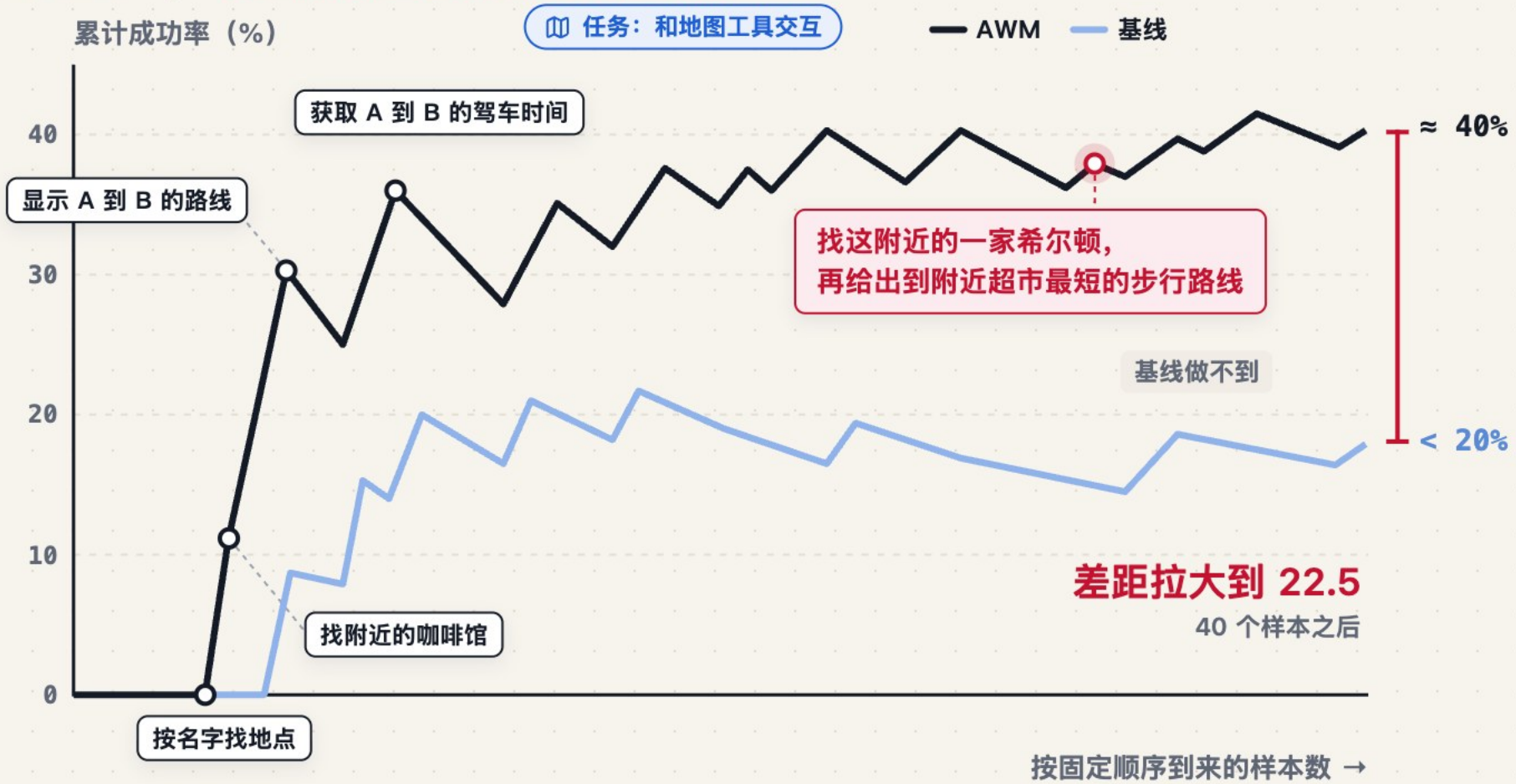
归纳提示词（AWM 附录 A.1，中文大意）：从多个网页任务里，提取公共的工作流

- ✓ 找多个任务间反复出现的动作子集
- ✓ 每个工作流至少两步
- ✓ 工作流彼此不要相似、重叠
- ✓ 不固定的部分换成有描述性的变量名



课件 p.26–27（中文重绘）

技能越攒越多，越做越复杂的任务



AWM（Wang et al., ICLR 2025）· 曲线按课件 p.28 原图形状重绘，读数按原图



课件 p.28（按原图重绘）

文字技能，还是代码技能?

文字 + 示例 · AWM 工作流

任务：在一个日期范围内搜索游戏配件

```
click(1274)   # 进入“电子游戏”分类
fill(473, {search_terms})   # 填入搜索词：商品名和年份
click(478)   # 执行搜索
```

某购物网站 · 页面改版后（示意）



agent 一步步自己发：`click(1274)` `fill(473, …)` `click(478)` `…`

+ 灵活：是个提示，agent 看着页面调整

– 手工：每个底层动作仍要自己发

代码 · ASI 技能

```
def search_product(search_box_id: str, query: str):
    """用搜索框搜索商品。
    参数：search_box_id 是搜索输入框的 ID，
③ 回车 ×
    示例：search_product('595', 'sony bluetooth headphones')
    """
    click(search_box_id)
    fill(search_box_id, query)
    keyboard_press('Enter')
```

+ 能当工具调用

+ 可测试：存之前先跑一遍

+ 有层次：技能调用技能

+ 高效：一次调用顶替很多步

– 死板 ① 点搜索框 ② 填词 ③ 回车 页面变了也照这三步走（示意）

✓ 脱离整条轨迹，单独生成测试用例

还能重构



课件 p.29（中文重绘）

归纳代码技能：ASI

裁判 → 归纳 → 准入

片段 一条成功的轨迹 → **归纳 LLM 写函数** → 重写轨迹 用新技能改写 → 执行 真的跑一遍 → 裁判 对不对？用没用？ → 记忆 工具库

片段 · 统计评论

查询：店里到目前为止收到的评论里，有多少条提到“satisfied”？

```
click("Marketing")
click("All Reviews")
fill(757, "satisfied")
click("Search")
send_msg_to_user("2")
```

改写后的轨迹

```
open_marketing_reviews()
search_reviews('757', '704', "satisfied")
send_msg_to_user("2")
```

裁判看执行结果

- 最终状态对不对？ ✓
- 技能用上了没有？ ✓

工具库 （示意）

之前攒下的…

之前攒下的…

`search_reviews`

`open_marketing_reviews`

回链：Voyager 在 Minecraft 里积累了一个代码技能记忆



课件 p.30（中文重绘）

代码技能可以单独测试：SkillWeaver

代码技能的准入 · SkillWeaver

```
# 提出候选技能
candidates = propose_skill(task)
# 拿它们练习做任务
episode = practice(candidates, task)
# 奖励模型看动作和截图
if reward_model(episode.actions, episode.screenshots, task):
    memory.add(candidate)          # 成了就存
else:
    candidate = revise(candidate, task)    # 不成就修改
```

药品网站 · 归纳出的识别药片技能

```
identify_pill(page, imprint="M366", color="White")
```

✓ 警告消失：color 用上了

```
+ if color:    # 给了颜色，就去找下拉框
+   await page.get_by_role("group",
+       name="Color and shape (optional)") \
+     .get_by_role("combobox",
+       name="Color (optional)") \
+     .select_option(color)   # 选上它
```

成了 → 存进记忆

不成 → 修改

linter 警告 | 奖励模型：看动作和截图 → 修改，写出新版本 → 再存进记忆

像 Reflexion 那样拿反馈改；代码虽然死板，也能这样测、这样修



课件 p.31（中文重绘）

代码技能还能压缩步数

任务（课件 p.32，中文重绘）

我刚搬家。能把账单地址改成“231 Willow Way, Suite 100，芝加哥，IL 60601”吗？然后把收货地址改成“987 Sycamore Circle，费城，PA 19102”。



有代码技能 4 步 快得多

从首页开始
第 1 步

```
navigate_to_address_settings('227', …)
```
一步到地址设置

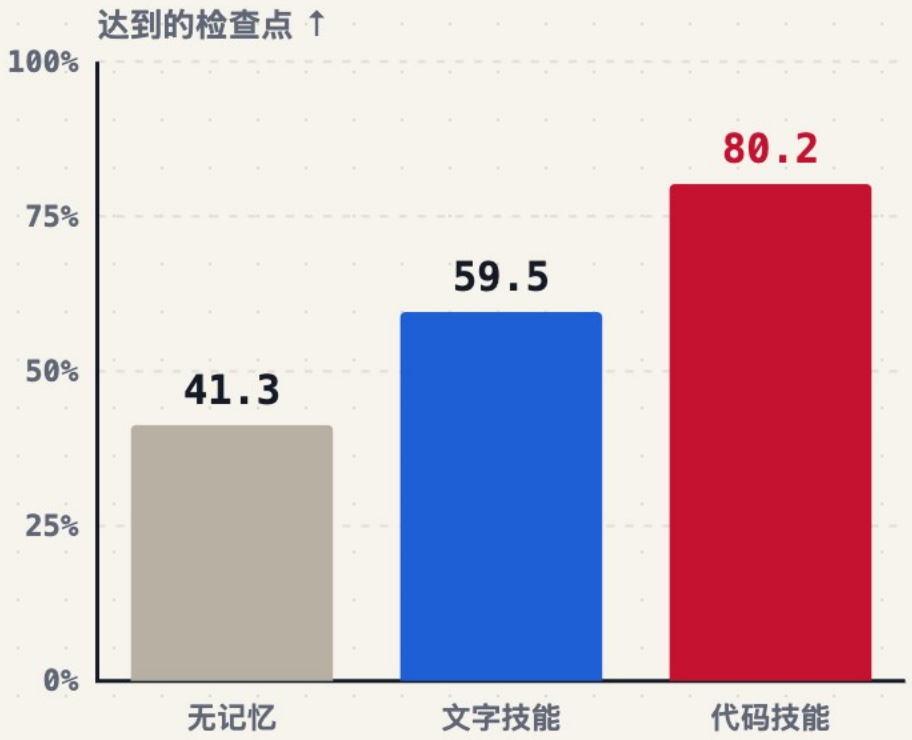
```
update_address_details('1724', …)
```
账单地址：芝加哥

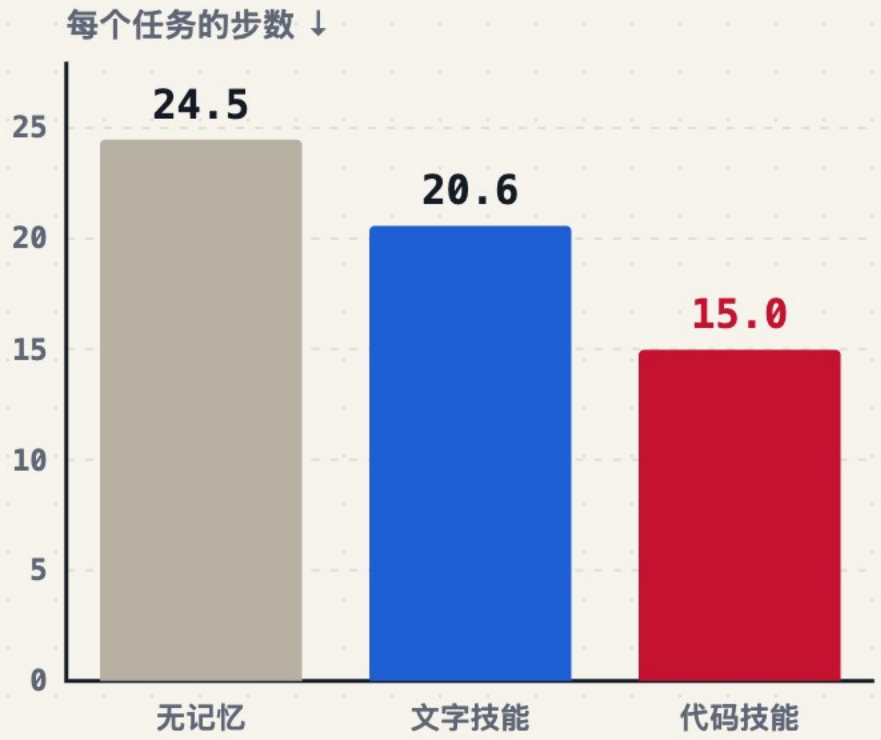
```
update_address_details('1724', …)
```
收货地址：费城



课件 p.32–33（中文重绘）

文字技能 vs 代码技能：两个指标





ASI（Wang et al., COLM 2025）表 2 · 五个 WebArena 网站平均



课件 p.32–33（中文重绘）

技能有多通用？换个网站就可能坏

任务：给我看最便宜的鞋子收纳。

target.com（页面为示意）



WebArena 上归纳 → 到 target.com 上评测

复用学到的技能

```
search_product('1724', 'shoe storage')
```

✓ 正确复用

排序技能 只适用于下拉菜单

```
def sort_listings(sort_dropdown_id, sort_option):
    """用排序下拉框给商品列表排序"""
    click(sort_dropdown_id)
    select_option(sort_dropdown_id, sort_option)
```

✕ 排序失败

🔁 让模型根据反馈去改技能



课件 p.34–35（中文重绘）

一个接口，多种实现

PolySkill 抽象类

```
# 抽象领域类：为购物网站定义一个“模式”
class AbstractShoppingSite:
    def search_product(self, query):
        """搜索商品"""
    def add_to_cart(self, item_id, quantity):
        """把指定商品加入购物车"""
    def checkout(self):
        """开始结账"""
    ## 组合技能
    def find_and_add_to_cart(self, query, item_id):
        self.search_product(query)
        self.add_to_cart(item_id)
    def purchase_item(self, query, item_id):
        self.find_and_add_to_cart(query, item_id)
        self.checkout()
```

只写一次

技能可以很多，同一时间只激活相关的部分

把软件工程的最佳实践用到 agent 上

亚马逊的实现

```
class AmazonWebsite(AbstractShoppingSite):
    def search_product(self, query):
        click(search_bar_id)
        fill(search_bar_id, query)
        keyboard_press('Enter')
```

Target 的实现 当前网站：只激活这一份

```
class TargetWebsite(AbstractShoppingSite):
    def search_product(self, query):
        fill(search_bar_id, query)
        click(search_button_id)
```

第一次碰到新网站（示意）

① 只有方法桩的抽象类 → ② 这个网站的实现

到了别的网站、旧实现不适用：再写一个实现



课件 p.34–35（中文重绘）

三种表示的取舍

表示	长处	短处
检索出的片段 留完整历史，按需检索	保住了具体行为	篇幅很长、难迁移
文字技能 提炼成结论、写成文字	指导灵活	提不了效率，每一步还要模型自己走
代码技能 写成代码	可执行、可组合、高效	可能很脆，环境一变就坏

上一讲留下的问题

- 完整历史
- 结论和手册
- 代码

各自什么时候出错？→ 这张表

多少留给模型，多少固化在产物里？

怎么结合最好，还有很多研究可做

← 留给模型临场决定

固化在产物里 →

Agent Skills 的技能包

`SKILL.md`

`scripts/`

两者结合



课件 p.36（中文重绘）

技能的生命周期：学、用、维护





课件 p.38（中文重绘）

从失败里学：ReasoningBank

生命周期 学 → 用 → 维护

任务（中文重绘）

告诉我索尼蓝牙耳机的完整型号，再给出在售型号的价格区间。

× 失败：什么都没发给用户

Bluetooth headphones Sony

5,578 条结果 · 每页 12 条

下一页 → 下一页 → 下一页 → 下一页 → 下一页 → 下一页 →

步数 ——— 上限

Bluetooth 或 headphones 或 Sony 词之间是"或"，不是"与"

索尼耳机 | 蓝牙音箱 | 索尼电视 | 耳机线 | 蓝牙鼠标 | 索尼相机

沾边的全列出来了（示意）

归纳出的文字策略

一 优化搜索词

搜索词太笼统，答案埋在几千条沾边结果里：翻页之前先收紧

二 调整每页显示的条数

每页 12 条，每点一次"下一页"就花一步，却看不到多少

三 用好现有的筛选

侧边栏的分类筛选，不翻页就把列表缩小

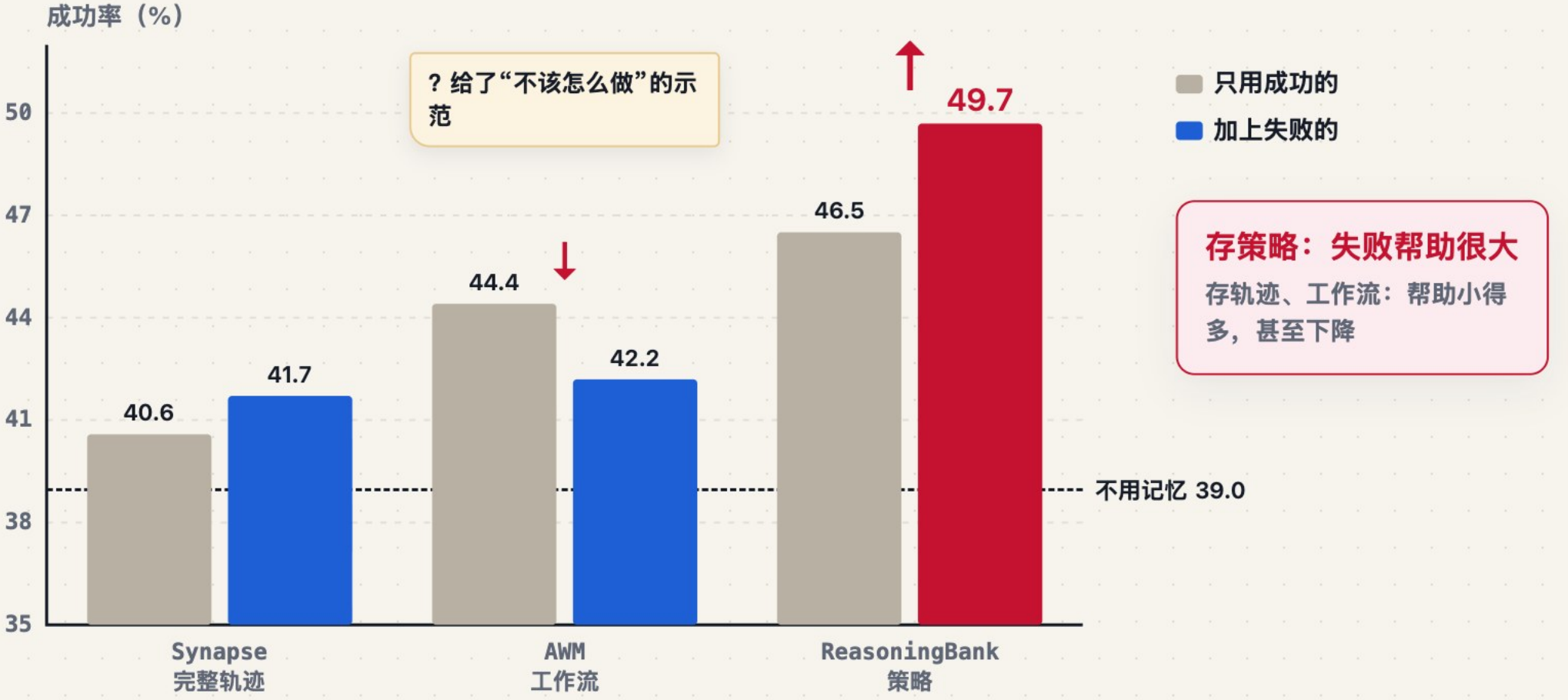
框架同 AWM、ASI · 失败的轨迹也能用



课件 p.38（中文重绘）

失败对策略有用，对轨迹和工作流用处小

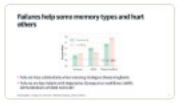
生命周期 学 → 用 → 维护



存策略：失败帮助很大

存轨迹、工作流：帮助小得多，甚至下降

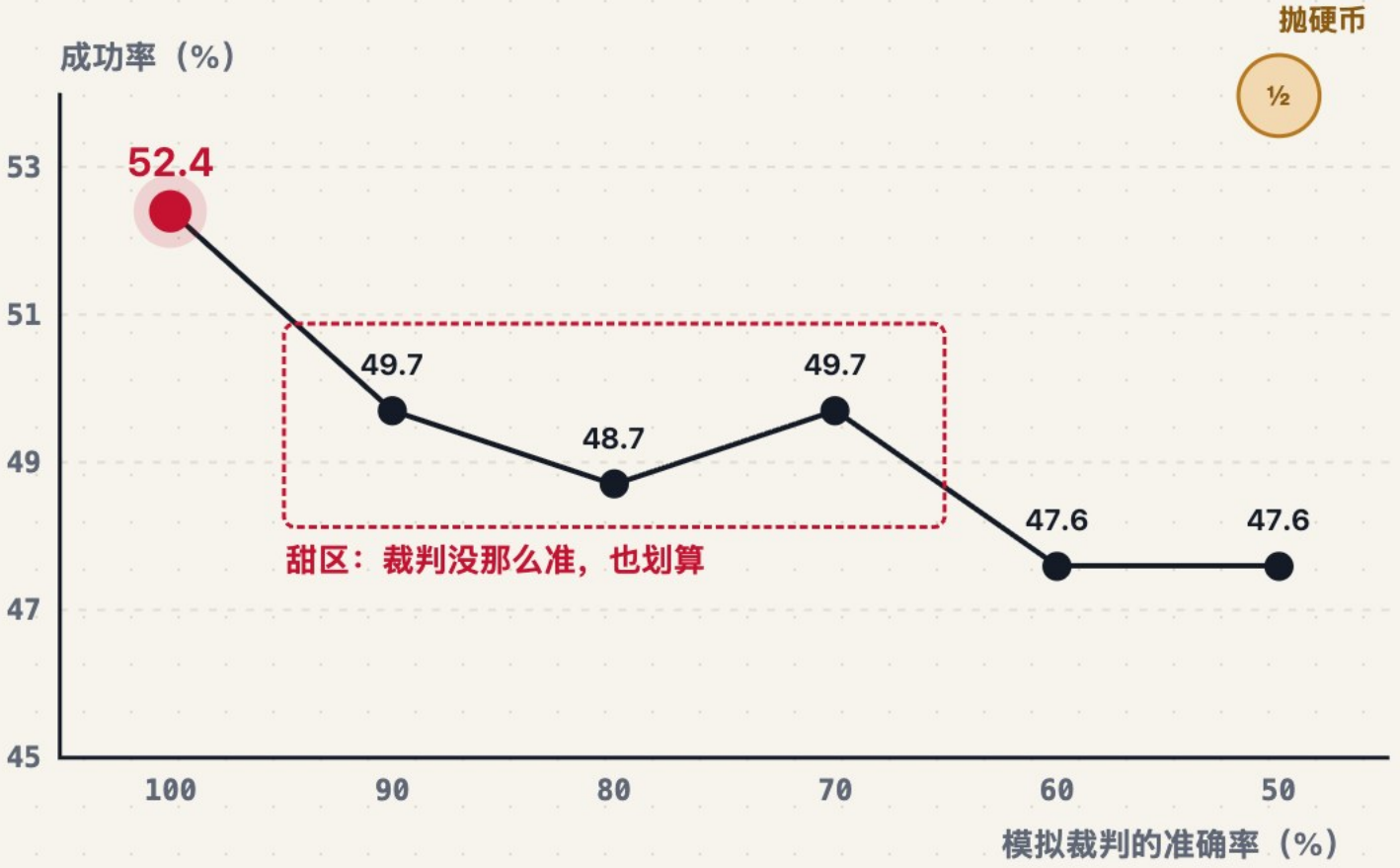
ReasoningBank（Ouyang et al., ICLR 2026）· WebArena 购物网站 · Gemini-2.5-flash



课件 p.39

裁判没那么准，也划算

生命周期 学 → 用 → 维护



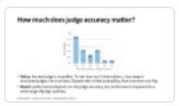
ReasoningBank（Ouyang et al., ICLR 2026）· 裁判校准分析

模型当裁判：看最终状态，判断对没对

并不完美

模拟裁判

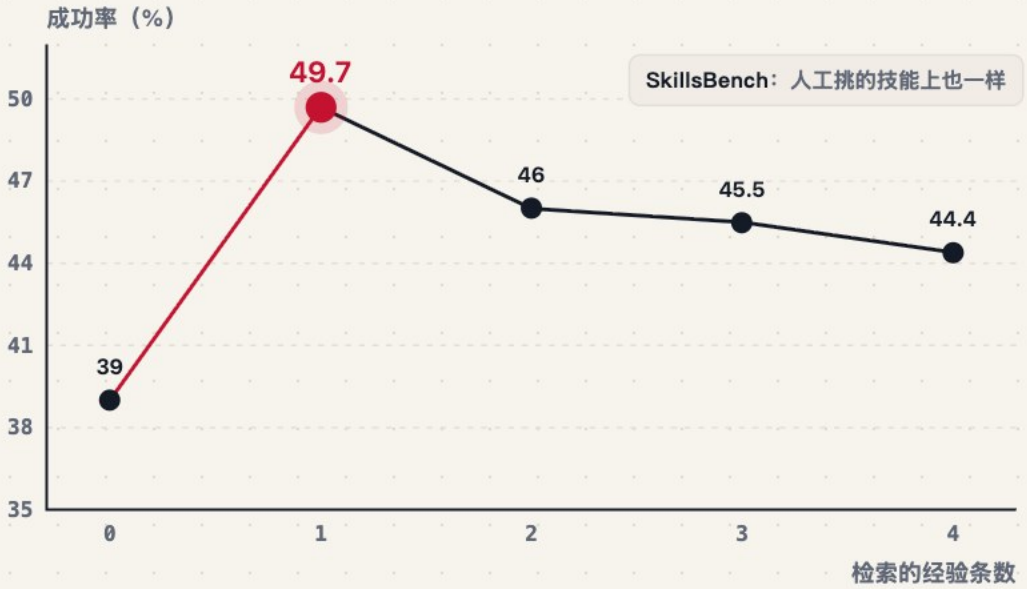
把真实标签按固定概率翻转：从全对，一直到抛硬币



课件 p.40

什么时候读回来：不是越多越好

生命周期 学 → 用 → 维护



ReasoningBank（Ouyang et al., ICLR 2026）· WebArena 购物网站

原因一 · 记忆

跑过的任务不够多，记忆里没有相关的：塞得越多越不相关。这是根本局限

原因二 · 模型

上下文里东西一多，就被无关的内容带偏

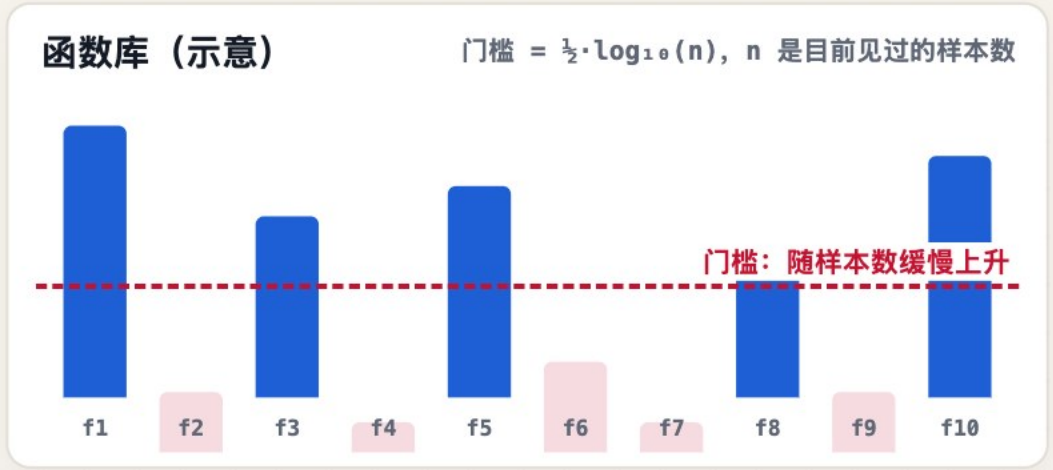
↑ 可以靠训练改善



课件 p.41

记忆整合：用得少就删

生命周期 学 → 用 → 维护

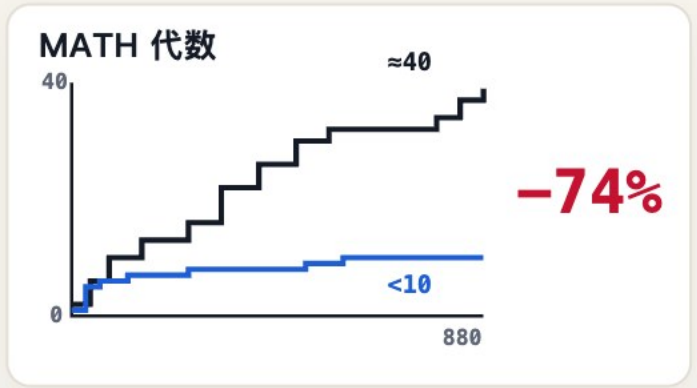


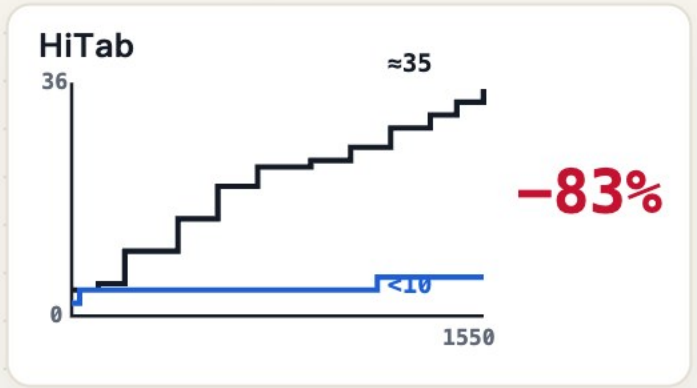
Not All Skills Help

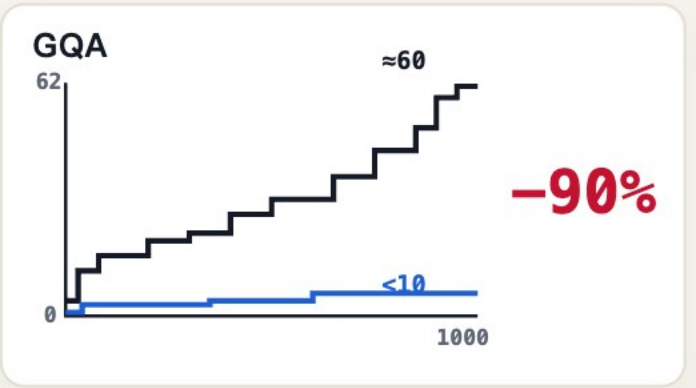
一个技能对一类任务有帮助，对另一类可能有害

很像缓存：用得少就丢

记忆小了，挑起来负担轻，效率高，成功率也可能提高









课件 p.42

值不值得存，要等复用时才知道

提示模型看经验 → 猜哪些有用 → 就存　　临时拼凑

训练

任务 1

把索尼耳机加进心愿单

写一个 search(query) 技能，运行

✓ 任务成功

→ 存

共享记忆

search(query)

✓ 被复用了，才知道值得

→ 取

任务 2

查无线键盘的价格区间

调用 search(query)

✓ 任务成功

SAGE 一次 rollout 里放两个相关任务　ACL 2026

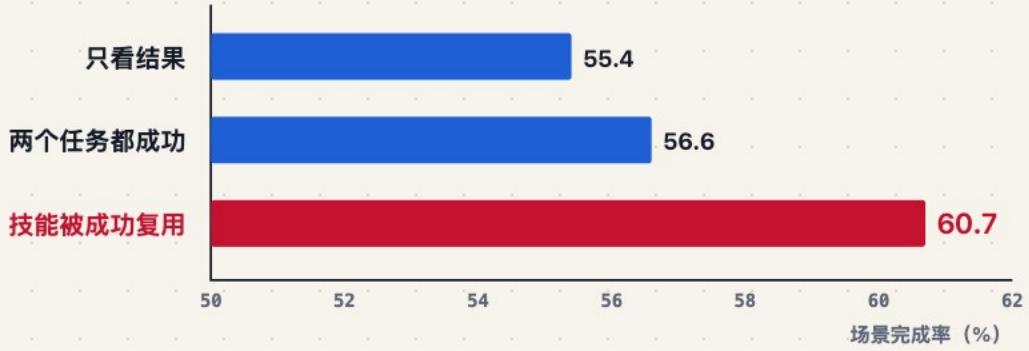
AgeMem 用一条很长的轨迹　ACL 2026

任务 1 的奖励 $r_1 + \mathbb{1}[\text{两个任务都成功}] \cdot \mathbb{1}[\text{任务 2 复用了它的技能}]$



课件 p.43（中文重绘）

奖励里鼓励复用，完成率更高



SAGE（Wang et al., ACL 2026）· AppWorld test-normal · Qwen2.5-32B · 横轴从 50 起

强化学习 —— 不可微也能套

存 → 取 → 用

再加一项：技能库尽量小 设想

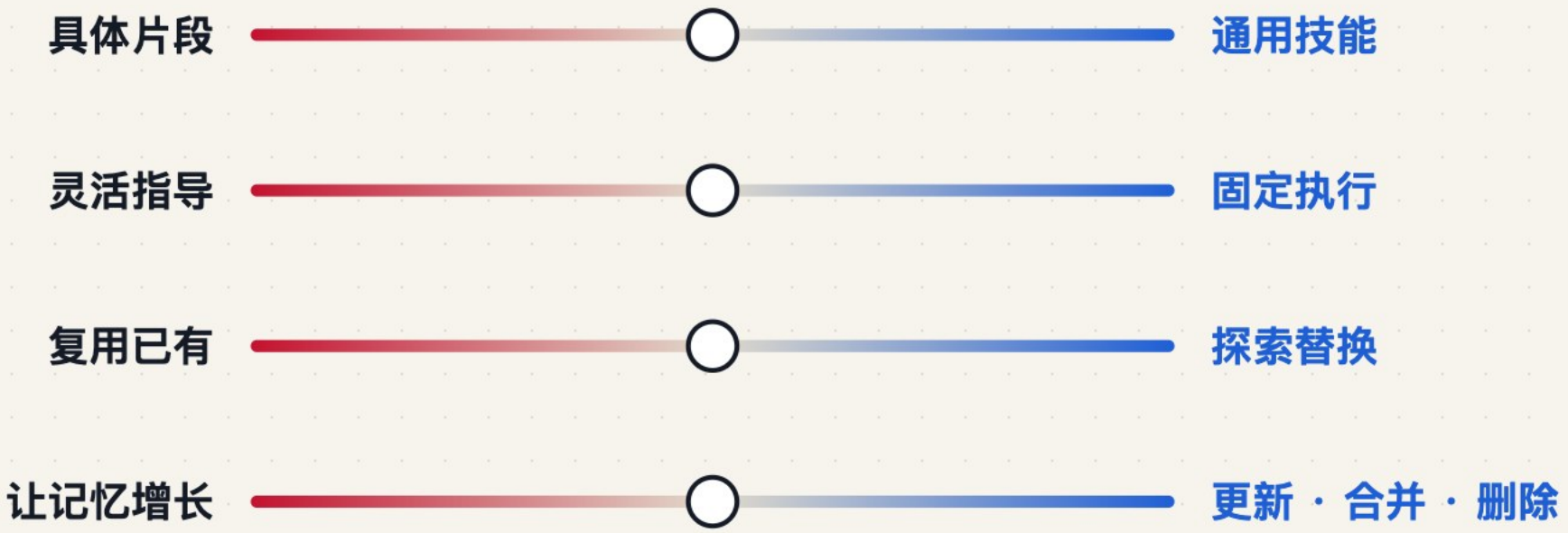
正则化：别过拟合





课件 p.43

背后是四组取舍





课件 p.44–46

每一步，归谁管

决策	人来做	agent 来做
编写和归纳	技能已知，人来写	只能在交互中摸索出来
验证和准入	对错是规范性的、事关重大	结果能执行、测试成本低
选择和使用	罕见例外，要结合情境	路由反复发生、能产生反馈
维护	要有人负责	漂移看得出来、也修得了



课件 p.44–46

留两个问题

1 你会把什么存成片段、事实、教训、文字技能，或者代码?

2 agent 什么时候该删掉一个技能，什么时候该改它?



课件 p.44–46

CMU 11-768 · AI AGENTS

第 4 讲 · 记忆和技能

下一讲：规划、任务拆解和多 agent 协作

每晚 20:00 更新



课件 p.44–46